Vytvořit na lokálním prostředí

```
git init --bare <cesta>
```

Filter by title

WARNING

- **Git**
  `<cesta>` = vytvoří úložiště do cesty, musí mít na konci cesty `.git`
  Úložiště (../git/repository.html)
- **Historie**

Použít do pracovního prostředí

```
git clone <cesta>
```

WARNING

`<cesta>` = adresa k úložišti, musí mít na konci cesty `.git`

NOTE

Může být lokální i online.

Klávesové zkratky

Parametry metody

Filter by title

- Informace o parametrech metody

`Ctrl + Shift + Space`

JetBrains (jetBrains.html)

Procházení seznamu:

- Vpřed

  `Ctrl + Shift + Space`

- Zpět

  `Ctrl + Shift + P`

XML komentáře

Zalomit řádek

- Použijte:

  `<para> </para>`

  > **WARNING**
  > `<para></para>` a `<br/>` nefungují pro zalomení řádku

  Příklad použití:

```
/// <summary>
///     This sentence shows up when the type is hovered
///     <para> </para>
///     <para>int PrimaryKey</para>
///     <para> </para>
///     <para>virtual Relation Relation</para>
/// </summary>
```

  Pro více informací: XML zalomení komentáře (https://stackoverflow.com/questions/7279108/how-to-add-a-line-break-in-c-sharp-net-documentation)

Vývojové metodiky

Techniky vývoje softwaru jsou postupy, které určují, jakým způsobem se vyvíjí software.

Filter by title

Techniky (techniques.html)

+ C# (net.html)

JavaScript (javascript.html)

Agilní metodika (Scrum)

- Zaměřuje se na spolupráci, zákaznickou spokojenost a schopnost reagovat na změny.
- V praxi tým pracuje v krátkých iteracích zvaných `sprinty`, které trvají obvykle **2-4 týdny**.

> **NOTE**
> Na konci každého sprintu tým prezentuje hotové funkce zákazníkovi a získává zpětnou vazbu.

> **WARNING**
> Není vhodná pro projekty, které vyžadují pevný plán a jasně definované výstupy.

1. Plánování sprintu (Sprint Planning)
   Týmová schůzka na začátku sprintu, která se koná jednou.

   Výběr úkolů (z backlogu) pro daný sprint k provedení.

   Diskutování o tom, jak budou tyto úkoly provedeny.

2. Vývoj
   Každodenní práce na úkolech, které byly vybrány během plánování sprintu.

   Každodenní týmová schůzka zvaná: `Daily Scrum`, kde se diskutuje o pokroku a případných překážkách na úkolech.

   Testování je součástí této sekce vývoje.

   > **NOTE**
   > Jakmile je úkol nebo funkce vyvinuta, je ihned testována, aby se zajistilo, že funguje správně a splňuje požadavky.

   > **TIP**
   > Toto průběžné testování umožňuje rychlé odhalení a opravu chyb.

3. Revize sprintu (Sprint Review)
   Týmová schůzka na konci sprintu, která se koná jednou.

   Prezentuje se práce zákazníkovi a získává se zpětná vazba.

   > **TIP**

Tato schůzka je také příležitostí k diskusi o tom, co se v sprintu povedlo a co ne.

4. Retrospektiva sprintu (Sprint Retrospective)
   Týmová schůzka obvykle hned po revizi sprintu.

   Diskutuje se o tom, co se v průběhu sprintu povedlo, co se nepovedlo a jak se mohou věci zlepšit v dalším sprintu.

Techniky (techniques.html)

+ C# (net.html)

JavaScript (javascript.html)

Vodopádová metodika

Každá fáze musí být dokončena, než může začít další.

WARNING

Délka každé fáze závisí na velikosti a složitosti projektu, počtu lidí v týmu a mnoha dalších faktorech.

1. Analýza požadavků
   Na začátku projektu se shromažďují a analyzují požadavky.

   NOTE

   Toto zahrnuje porozumění potřebám zákazníka a definování, co bude systém dělat.

2. Návrh
   Vytváří se podrobný plán toho, jak bude systém fungovat a jak bude vypadat.
3. Implementace

   Návrh se převede na zdrojový kód.

   Programátoři píší kód, který realizuje návrh systému.
4. Testování

   Kontroluje se, zda systém splňuje požadavky definované v první fázi a zda neobsahuje chyby.
5. Nasazení
   Jakmile je systém otestován a schválen, je nasazen do produkčního prostředí.
6. Údržba
   Sleduje se výkon systému, opravují se chyby a přidávají se nové funkce podle potřeb zákazníka.

Kanban

Vizuální systém řízení práce, který se zaměřuje na dokončení úkolů.

NOTE

Zaměřuje se na průběžnou dodávku a minimalizaci času stráveného na úkolech.

WARNING

Není vhodný pro projekty, které vyžadují pevné plány a termíny

1. Definování úkolů
   Na začátku se vytvoří seznam úkolů, které je třeba dokončit.

   > **TIP**
   > Tyto úkoly se zapisují na karty, které se umístí na Kanban tabuli.

2. Vizualizace práce
   Kanban tabule je rozdělena na několik sloupců, které reprezentují různé stavy úkolu.

   > **TIP**
   > Například: `Backlog` , `Todo` , `Done` , atd...

   Karty s úkoly se přesouvají mezi těmito sloupci podle toho, v jakém stavu se nacházejí.

3. Práce na úkolech
   Tým začne pracovat na úkolech, začínajíc u těch, které jsou nejvíce prioritní.

   > **NOTE**
   > Jakmile je úkol dokončen, karta se přesune do dalšího sloupce.
   >
   > Například sloupec: `Done`

4. Omezení práce v průběhu (Work in Progress, WIP)
   Kanban klade důraz na omezení množství práce, která se může dělat současně.

   > **NOTE**
   > Zvyšuje se efektivita a snižuje se doba, kterou úkol stráví v systému.

5. Průběžné zlepšování
   Tým pravidelně hodnotí svůj proces a hledá způsoby, jak ho zlepšit.

   > **TIP**
   > To může zahrnovat změnu počtu úkolů, prioratizaci úkolů atd...

Rychlé prototypování

Proces pro vytvoření funkčního modelu projektu co nejrychleji, aby bylo možné testovat a iterovat nápady.

> **NOTE**

V kontextu Unity, to znamená vytvoření základní verzi hry nebo aplikace, která zahrnuje pouze klíčové mechaniky a funkce.

> **NOTE**
>
> Rychlá iterace
>
> - Rychle testovat a provádět nápady.
> - Pokud vytvoříte prototyp, zkuste ho co nejdříve otestovat a získat zpětnou vazbu.
> - Potom můžete na základě této zpětné vazby upravit a vylepšit svůj prototyp.

Techniky (techniques.html)

+ C# (net.html)

JavaScript (javascript.html)

> **IMPORTANT**
>
> **Cílem je vytvořit funkční model** vašeho projektu, **ne dokonalý produkt**.
>
> Nebojte se udělat kompromisy v kvalitě, pokud to znamená, že můžete rychleji testovat a iterovat své nápady.

Postup

1. Definice svého konceptu
   Než se začne s prototypováním, měli byste mít jasnou představu o tom, co chcete vytvořit.

   > **TIP**
   >
   > To může zahrnovat definování klíčových mechanik, funkcí a cílů vašeho projektu.

2. Vytvořit základní scénu v Unity

   > **TIP**
   >
   > Tato scéna bude sloužit jako základ pro váš prototyp.

3. Přidat základní objekty
   Přidejte do scény základní objekty, jako jsou krychle, koule a válce, které můžete použít k reprezentaci různých prvků ve vaší hře.

4. Přidejte mechaniky a funkce
   Použijte skriptování a vestavěné nástroje Unity k přidání mechanik a funkcí do vašeho prototypu.

5. Testujte a iterujte
   Jakmile máte základní prototyp, začněte ho testovat.

   > **NOTE**
   >
   > Získejte zpětnou vazbu od ostatních a na základě této zpětné vazby upravte a vylepšujte svůj prototyp.

6. Opakujte proces
   Po provedení změn na svém prototypu ho znovu otestujte a pokračujte v tomto cyklu, dokud nejste spokojeni s výsledkem.

> **TIP**
> Rychlé prototypování je iterativní proces.

Techniky (techniques.html)

Pojmenování BEM

+ C#/.net (net.html)

BEM = "Block Element Modifier"

JavaScript (javascript.html)

Metodika pro pojmenování tříd v `HTML` a `CSS`.

> **NOTE**
> Pomáhá udržet váš kód organizovaný a snadno pochopitelný, a to i pro ostatní vývojáře, kteří se na váš kód podívají.

Příklad:

```
<div class="block"> <!-- Block -->
  <div class="block__element"> <!-- Element -->
  </div>
  <div class="block__element--modifier"> <!-- Element with modifier -->
  </div>
</div>
```

```
.block { ... }
.block__element { ... }
.block__element--modifier { ... }
```

Block

Jedná se o samostatnou entitu, která má smysl sama o sobě.

Například: `header`, `container`, `menu`, `checkbox`, atd.

Element

Část bloku, která nemá samostatný význam.

Je semanticky vázána na svůj blok.

Například: `menu item`, `list item`, `checkbox caption`, atd.

Modifier

Varianta bloku nebo prvku, která mění vzhled nebo chování.

Například: `disabled`, `highlighted`, `checked`, atd.

Syntax BEM je poté následující:

- Názvy bloků a elementů jsou odděleny dvěma podtržítky: `__`
- Názvy modifikátorů jsou odděleny dvěma pomlčkami: `--`

Techniky (techniques.html)

+ C# (net.html)

JavaScript (javascript.html)

Základy pojmenování při vývoji

Jasné a popisné názvy

Názvy proměnných, funkcí, tříd atd. by měly být dostatečně popisné, aby bylo jasné, co dělají nebo co reprezentují.

Například:

Místo: `p`

použijte: `product`

místo: `calc`

použijte: `calculateAverage`.

> **TIP**
> Vyhněte se používání zkratek a akronymů.

Boolovské proměnné

Boolovské proměnné by měly začínat předponou jako: `is`, `has`, `can` atd., které naznačují, že hodnota může být pravdivá nebo nepravdivá.

Například:

`isAvailable`, `hasPermission`, `canExecute`.

- Názvy funkcí

  Funkce by měly začínat slovesem, které popisuje, co funkce dělá.

  Například:

  `getUserName()`, `calculateTotalPrice()`, `printReport()`.

- Konzistentní názvosloví

  Pokud se rozhodnete pro určitý styl názvosloví, buďte konzistentní v jeho používání po celém kódu.

  Například:

Pokud používáte `camelCase` pro názvy proměnných, používejte je všude.

Jednotné názvy pro stejné typy proměnných

Pokud máte více polí.

Například:

`firstArray` , `secondArray` .

Množné čísla (plurál) pro kolekce

Pokud proměnná ukládá kolekci objektů, použijte množné číslo.

Například:

`users` , `products` , `items` .

Konstanty pro "magické" hodnoty

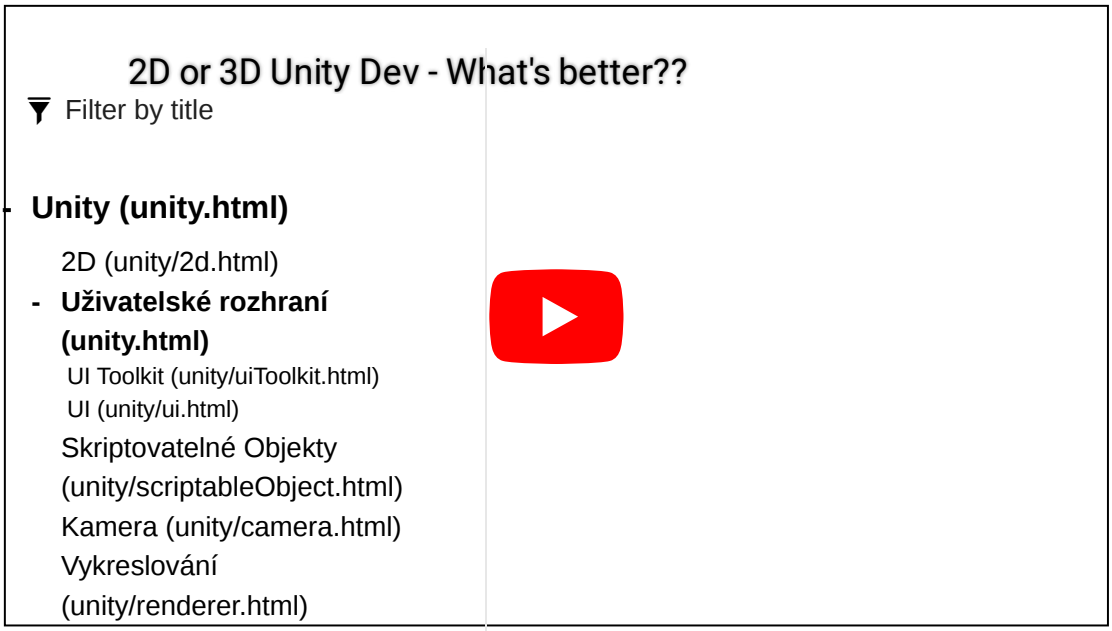
Pokud kód obsahuje "magické" hodnoty, které nejsou okamžitě zřejmé, zvažte jejich nahrazení pojmenovanými konstantami.

Například:

místo: `if (status == 1)`

použijte: `if (status == STATUS_ACTIVE)` .

Výběr hry 2D či 3D



Rychlé prototypování

Klíčová slova

2 Keywords I didn't fully understand when starting gamedev...

- **Unity (unity.html)**
  - 2D (unity/2d.html)
  - **Uživatelské rozhraní (unity.html)**
    - UI Toolkit (unity/uiToolkit.html)
    - UI (unity/ui.html)
  - Skriptovatelné Objekty (unity/scriptableObject.html)
  - Kamera (unity/camera.html)
  - Vykreslování (unity/renderer.html)
  - Navigace (Pohyb) (unity/navigation.html)

Vývojové vzory

3 Game Programming Patterns WE ACTUALLY NEED.

Co je MongoDB

= Dokumentová databáze

- NoSQL (typ databáze)

Filter by title

Mongo databáze (mongoDB.html)

> **NOTE**
> Nevyužívá tabulkový formát, který je běžný u SQL databází.

- Data ukládá ve formátu zvaném **`BSON`** . (Binární verze formátu `JSON`)

> **TIP**
> Formát `BSON` podporuje více datových typů.
>
> Efektivnější při encoding a decoding než `JSON` .

Collections

- Data jsou organizována do **`collections`**

> **NOTE**
> Jsou ekvivalentem tabulek v SQL.

- Každá tato kolekce obsahuje **`documents`** , což jsou jednotlivé záznamy dat.

> **NOTE**
> Na rozdíl od řádků v tabulce SQL nemusí mít dokumenty v MongoDB stejnou strukturu.
>
> To znamená, že různé dokumenty ve stejné kolekci mohou mít různé sady polí.
>
> Příklad:
>
> ```
> var document1 = { name: "Peter", age: 30, residence: "Prague" };
> db.myPeople.insert(document1);
>
> var document2 = { name: "Anna", age: 25, occupation: "Engineer" };
> db.myPeople.insert(document2);
> ```

Klíčové pojmy

Dokumenty

Záznamy v MongoDB.

Mongo databáze (mongoDB.html)

> **NOTE**
> Každý dokument je struktura dat podobná JSON.

Kolekce

Skupiny dokumentů.

> **NOTE**
> Jsou to ekvivalenty tabulek v SQL.

BSON

Formát ve kterém jsou data uložena.

> **NOTE**
> Je to binární verze JSON.

Kód

Vytvořit

- Databázi

```
use mydb
```

- Kolekci

```
mydb.createCollection('mycollection')
```

- Vložit dokument do kolekce

```
mydb.mycollection.insert({name: 'test'})
```

- Vložit více dokumentů do kolekce

```
mydb.mycollection.insertMany([{name: 'test1'}, {name: 'test2'}])
```

- Vytvořit index

```
mydb.mycollection.createIndex({name: 1})
```

Mongo databáze (mongoDB.html)

TIP

Index je vytvořen na pole `name`

Vzestupný = 1

Sestupný = -1

Vzestupné a sestupné indexy určují pořadí, ve kterém jsou data v indexu uložena, což může ovlivnit výkon a rychlost dotazů, které vyžadují řazení

- Vytvoření více indexů

```
mydb.mycollection.createIndexes([{ key: { name: 1 } }, { key: { age: -1 } }])
```

Hledat

- Výpis databází

```
show dbs
```

- Výpis dokumentů

```
mydb.mycollection.find()
```

- Výpis kolekcí

```
show collections
```

- Hledání dokumentu

```
mydb.mycollection.find({name: 'test'})
```

- Hledání dokumentu s určitými poli

```
mydb.mycollection.find({name: 'test'}, {name: 1})
```

- Hledání dokumentu s regulárním výrazem

```
mydb.mycollection.find({name: {$regex: 'te.*'}})
```

Mongo databáze
(mongoDB.html)

Aktualizovat

- Aktualizace dokumentu

```
mydb.mycollection.update({name: 'test'}, {$set: {name: 'newTest'}})
```

- Aktualizace více dokumentů

```
mydb.mycollection.updateMany({}, {$set: {name: 'newTest'}})
```

- Aktualizace dokumentu s upsert

```
mydb.mycollection.update({name: 'test'}, {$set: {name: 'newTest'}}, {upsert: tr
ue})
```

> **NOTE**
> "upsert" **je kombinací "update" a "insert"**
> Aktualizuje existující dokument, nebo pokud dokument neexistuje, vloží nový dokument.

Smazat

- Smazání databáze

```
db.dropDatabase()
```

- Smazání kolekce

```
mydb.mycollection.drop()
```

- Smazání dokumentu

```
mydb.mycollection.remove({name: 'test'})
```

- Smazání všech dokumentů

```
mydb.mycollection.remove({})
```

Mongo databáze (mongoDB.html)

Počet

- Počet dokumentů v kolekci

```
mydb.mycollection.count()
```

- Počet dokumentů odpovídajících určitému dotazu

```
mydb.mycollection.count({name: 'test'})
```

- Počet unikátních hodnot v určitém poli

```
mydb.mycollection.distinct('name').length
```

- Počet dokumentů odpovídajících regulárnímu výrazu

```
mydb.mycollection.count({name: {$regex: 'te.*'}})
```

Setřídit

- Seřazení dokumentů podle pole

```
mydb.mycollection.find().sort({name: 1})
```

- Seřazení dokumentů podle více polí

```
mydb.mycollection.find().sort({name: 1, age: -1})
```

- Seřazení a omezení počtu dokumentů

```
mydb.mycollection.find().sort({name: 1}).limit(5)
```

- Seřazení a přeskočení dokumentů

```
mydb.mycollection.find().sort({name: 1}).skip(5)
```

Rady a Tipy

Mongo databáze (mongoDB.html)

Povolení autorizace

- MongoDB má vestavěný systém pro správu uživatelů a rolí.

  Pro povolení autorizace upravte konfigurační soubor MongoDB a nastavte `security.authorization` na `enabled`.

```
security:
  authorization: "enabled"
```

Využití indexů

- Indexy v MongoDB vytváříte pomocí metody `createIndex()`.

  Například pro vytvoření vzestupného indexu na pole `name` v kolekci `mycollection` použijete následující příkaz:

```
db.mycollection.createIndex({name: 1})
```

Optimalizace dotazů

- MongoDB poskytuje operátor `explain()`, který vám umožní zjistit, jak databáze vykonává váš dotaz.

  Tímto způsobem můžete identifikovat, které části dotazu je třeba optimalizovat.

```
db.mycollection.find({name: 'test'}).explain()
```

Správné modelování dat

- MongoDB je dokumentová databáze, která umožňuje velmi flexibilní modelování dat.

  Při návrhu vašeho datového modelu zvažte, jak budou data dotazována a jaké budou pracovní zátěže.

Škálování

- MongoDB podporuje horizontální škálování pomocí replikačních sad a sharding.

Pro větší aplikace zvažte použití těchto funkcí pro zlepšení výkonu a dostupnosti.

Paměť

- MongoDB využívá paměť pro ukládání pracovní sady, což zlepšuje výkon dotazů.
- Ujistěte se, že váš server má dostatek RAM pro vaše pracovní zátěže.

Šetření prostředky

MongoDB databáze (mongoDB.html)

- Pokud máte dotazy, které se často opakují, zvažte ukládání výsledků těchto dotazů pro pozdější použití.

  To může šetřit výpočetní prostředky a zlepšit výkon vaší aplikace.

Izoluje aplikace se všemi jejími knihovnami, konfiguračními soubory a dalšími závislými soubory do kontejnerů.

> **NOTE**
> Kontejnery zajišťují, že aplikace mohou být spuštěny v jakémkoli prostředí.

Filter by title

Docker (docker.html)

Kubernetes (kubernetes.html)

Docker se stará o celý životní cyklus kontejnerů.

Kontejner → Vytvoření → Spuštění → Zastavení

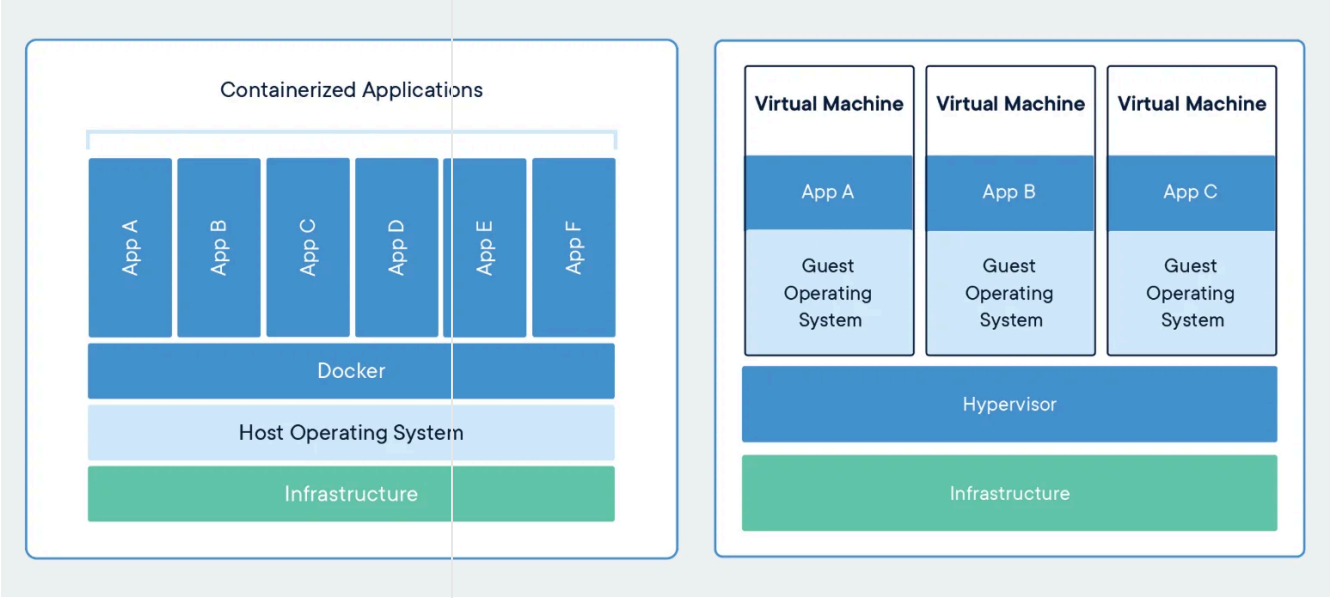
> **TIP**
> - Kontejnerizace je virtualizací jádra operačního systému.

  Všechny kontejnery běží v rámci jednoho operačního systému a sdílejí paměť, knihovny a další zdroje.

- Zdroje se tímto způsobem využívají efektivnější než klasická virtualizace.

  Spuštění kontejneru je navíc mnohem rychlejší než spuštění virtuálního stroje s instalací operačního systému.

- Malá režie a na stejném hardwaru můžete spustit více docker kontejnerů než virtuálních strojů



Základní pojmy

dockerfile

Textový soubor s instrukcemi k vytvoření `Docker image`.

Specifikuje operační systém, na kterém bude běžet kontejner, jazyky, lokace, porty a další komponenty.

docker image

Komprimovaná, samostatná část softwaru vytvořená příkazy v `Dockerfile`.

Je to "šablona" (aplikace plus požadované knihovny a binární soubory) potřebná k vytvoření a spuštění Docker kontejneru.

docker run

Příkaz, který spouští kontejnery.

> **NOTE**
>
> Každý kontejner je instancí jednoho image.

Docker (docker.html)

Kubernetes (kubernetes.html)

docker hub

Oficiální úložiště pro sdílení `docker image`.

> **TIP**
>
> Obsahuje officiální `docker image` z open-source projektů a neoficiální od komunity.
>
> Možnost pracovat i s lokálními docker úložišti.

docker engine

Jádro softwaru docker.

Technologie na principu klient-server, která vytváří a provozuje kontejnery.

docker compose

Definice ke spuštění více kontejnerů.

Příkazy

Stáhnout docker image

```
docker pull [Image name]
```

> **NOTE**
>
> [Image name]
>
> Je název `docker image`.
>
> Například: `mcr.microsoft.com/dotnet/core/sdk:3.1`

Umístění docker image po stažení:

- Linux:

```
/var/lib/docker/
```

- Windows:

```
C:\ProgramData\DockerDesktop
```

- macOS:

```
~/Library/Containers/com.docker.docker/Data/vms/0/
```

Docker (docker.html)

Kubernetes (kubernetes.html)

Sestavení docker image

- `docker build [-t] customFolder`

  Sestaví kontejner pro docker image ve vybraném adresáři.

  > **NOTE**
  >
  > `customFolder`
  >
  > Název vybraného adresáře k sestavení docker image.
  >
  > Může být například aktuální adresář: `.` , nebo jakkoli jinak.
  >
  > `-t`
  >
  > Pojmenování image a tagu. (Pokud není zadán parametr -t, použije se tag: `latest`)

- Příklad:

```
docker build -t myapp .
```

  > **TIP**
  >
  > `myapp`
  >
  > Název pro nově sestavený kontejner. (Může být jakýkoli.)
  >
  > `.`
  >
  > Pracovní adresář v dockeru. (V tomto příkladu kořenový adresář.)

Spuštění kontejneru z docker image

- `docker run [docker image]`

  Spustí kontejner pro docker image.

```
docker run kitematic/hello-world-nginx
```

TIP

Spustí docker kontejner s docker image: `kitematic/hello-world-nginx`

Docker (docker.html)

Kubernetes (kubernetes.html)

Spustit na jiném portu

```
docker run -p 70:80 kitematic/hello-world-nginx
```

NOTE

`-p`

Mapuje port 70 na hostitelském stroji na port 80 uvnitř kontejneru. (To znamená, že pokud aplikace uvnitř kontejneru poslouchá na portu 80, bude přístupná na portu 70 hostitelského stroje.)

`kitematic/hello-world-nginx`

Název docker image ke spuštění.

Spustit v interaktivním módu

```
docker run -it kitematic/hello-world-nginx
```

NOTE

Užitečné, pokud chcete spustit kontejner a poté v něm spustit další příkazy, například při ladění nebo vývoji.

Odstranit po zastavení

```
docker run --rm kitematic/hello-world-nginx
```

NOTE

`--rm`

Docker automaticky odstraní kontejner, když je běh kontejneru přerušení.

TIP

Užitečné, pokud nechcete, aby se vaše lokální úložiště naplnilo zastavenými kontejnery.

Spuštění více kontejnerů z docker image najednou

NOTE

Musíte použít soubor YAML k definování služeb vaší aplikace.

Dockeru Compose (docker.html) po příkazu `docker-compose up` můžete vytvořit a spustit všechny služby definované ve vašem souboru `docker-compose.yml`.

Kubernetes (kubernetes.html)

TIP

Automaticky použije lokální `docker image`, pokud je k dispozici.

Příklad souboru `docker-compose.yaml`:

```
# Verze Docker Compose souboru
version: '3.4'

# Definice služeb
services:
  # Název služby
  webapp:
    # Obraz, který se má použít pro tuto službu
    # Tento obraz je vzorová aplikace ASP.NET Core od Microsoftu
    image: mcr.microsoft.com/dotnet/core/samples:aspnetapp
    # Instrukce pro sestavení obrazu
    build:
      # Kontext pro sestavení, obvykle je to adresář obsahující Dockerfile
      context: .
      # Cesta k Dockerfile
      dockerfile: Dockerfile
    # Mapování portů mezi hostitelem a kontejnerem
    # Formát je "host:kontejner"
    # Toto nastavení říká Dockeru, aby přesměroval port 8000 na hostiteli na port 80
v kontejneru
    ports:
      - "8000:80"
```

Dockerfile

V dockeru **není žádná výchozí složka**.

TIP

Když vytváříte Dockerfile, můžete nastavit pracovní adresář v kontejneru pomocí příkazu `WORKDIR` .

TIP

Pokud není nastaven `WORKDIR` , vztahuje se vše na kořenový adresář (/) kontejneru.

Příklad pro .NET Core

Kubernetes (kubernetes.html)

```
# Používáme oficiální .NET Core runtime image z Docker Hub
# 'dotnet' je jméno image a '3.1' je tag, který specifikuje verzi
FROM mcr.microsoft.com/dotnet/core/runtime:3.1

# Nastavíme pracovní adresář v kontejneru na /app
# Pokud tento adresář neexistuje, docker ho vytvoří
WORKDIR /app

# Kopírujeme výstup buildu z našeho stroje do kontejneru
# 'publish' je cesta k výstupu buildu na našem stroji
# '.' znamená aktuální (pracovní) adresář v kontejneru
COPY ./publish .

# Nastavíme spustitelný soubor pro kontejner
# 'myapp.dll' je název naší aplikace
ENTRYPOINT ["dotnet", "myapp.dll"]
```

NOTE

Vytvoří `docker image` pro vaši aplikaci .NET Core.

Když spustíte kontejner z této image, vaše aplikace se automaticky spustí.

Příklad pro C# Aplikaci

```
# Používáme oficiální .NET Core SDK image z Docker Hub
# 'dotnet' je jméno image a '3.1' je tag, který specifikuje verzi
FROM mcr.microsoft.com/dotnet/core/sdk:3.1

# Nastavíme pracovní adresář v kontejneru na /app
# Pokud tento adresář neexistuje, docker ho vytvoří
WORKDIR /app

# Kopírujeme všechny soubory z našeho stroje do kontejneru
# '.' znamená aktuální adresář na našem stroji
# '.' znamená aktuální (pracovní) adresář v kontejneru
COPY . .

# Spustíme příkaz 'dotnet restore', který stáhne všechny potřebné NuGet balíčky
RUN dotnet restore

# Spustíme příkaz 'dotnet publish', který vytvoří výstup buildu naší aplikace
RUN dotnet publish -c Release -o out

# Nastavíme spustitelný soubor pro kontejner
# 'myapp.dll' je název naší aplikace
ENTRYPOINT ["dotnet", "out/myapp.dll"]
```

Docker (docker.html)

Kubernetes (kubernetes.html)

NOTE

Tento Dockerfile vytvoří docker image pro vaši aplikaci C#.

Když spustíte kontejner z této image, vaše aplikace se automaticky spustí.

Příklad .NET Core a lokálních NuGet balíčků

```
# Používáme oficiální .NET Core SDK image z Docker Hub
FROM mcr.microsoft.com/dotnet/core/sdk:3.1

# Nastavíme pracovní adresář v kontejneru na /app
WORKDIR /app

# Kopírujeme všechny soubory z našeho stroje do kontejneru
COPY . .

# Spustíme příkaz 'dotnet restore', který načte všechny potřebné NuGet balíčky z lok
álního úložiště
# Předpokládáme, že všechny potřebné NuGet balíčky jsou uloženy v adresáři 'nuget' n
ašeho projektu
RUN dotnet restore --source ./nuget

# Spustíme příkaz 'dotnet publish', který vytvoří výstup buildu naší aplikace
RUN dotnet publish -c Release -o out

# Nastavíme spustitelný soubor pro kontejner
ENTRYPOINT ["dotnet", "out/myapp.dll"]
```

Docker (docker.html)

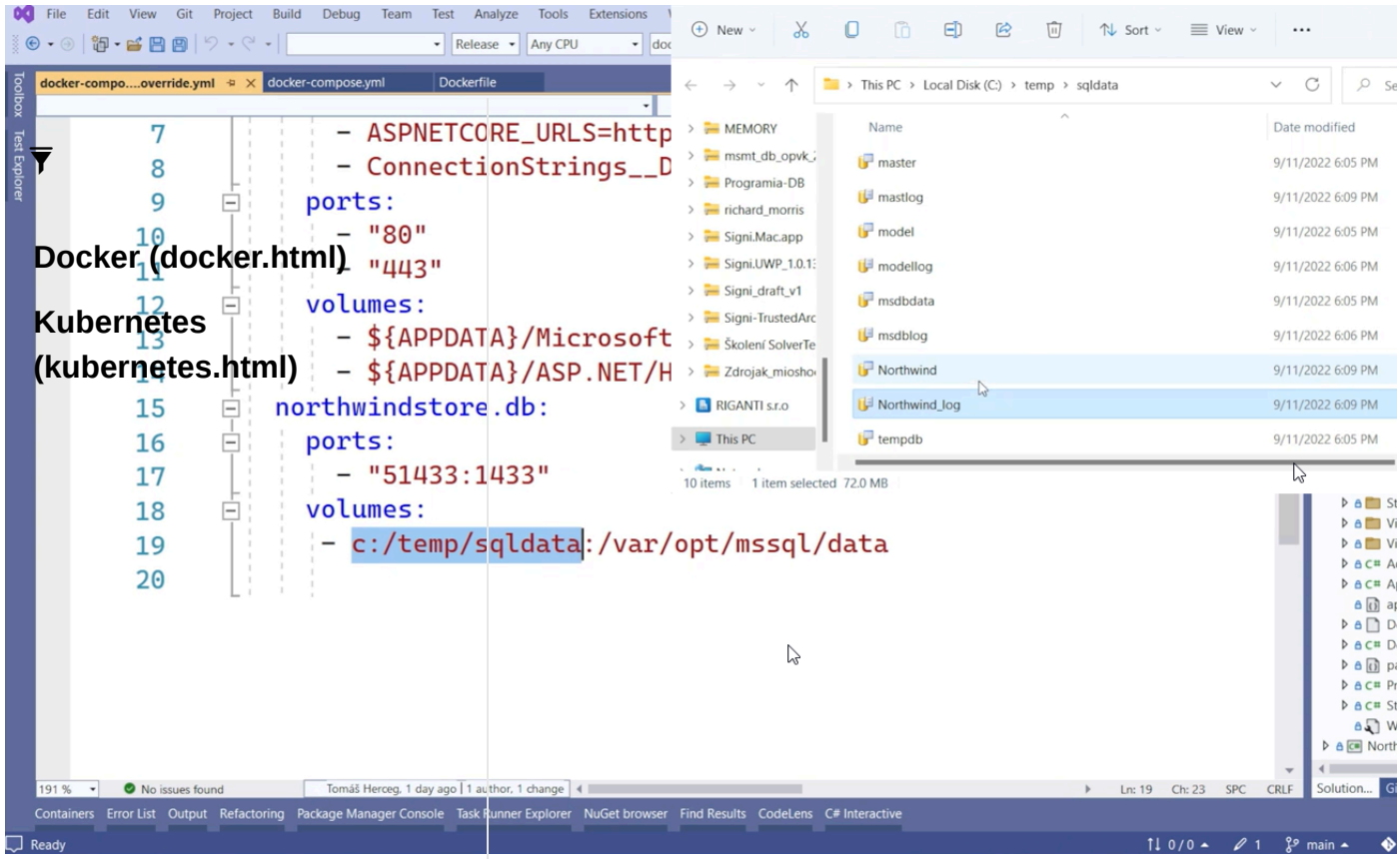
Kubernetes (kubernetes.html)

TIP

V tomto příkladu předpokládáme, že všechny potřebné NuGet balíčky jsou uloženy v adresáři `nuget` vašeho projektu.

Příkaz `dotnet restore --source ./nuget` pak načte tyto balíčky z lokálního úložiště místo stahování z internetu.

Zachování dat z kontejneru na lokálním disku



Zdroj (https://www.master.cz/blog/docker-kubernetes-kontejnery-jak-funguji-proc-je-chtit/)

Použití mřížky (grid)

1. View -> Show Grid (zobrazí mřížku)
2. View -> Grid and Axis... (nastavení mřížky)

Filter by title

Výpočet obrázku pro zarovnání na střed

Affinity software

(affinity.html)

Ukázka níže používá tyto hodnoty:

gridWidth = 256 px

gridHeight = 256 px

offsetX (posune obrázek do prava od původního) = 256 px

Univerzální vzorec:

Pro obecný posun o offsetX pixelů doprava od původní pozice:

1. **Původní souřadnice:**
   - $\text{originX} = \left(\frac{\text{gridWidth}}{2} - \frac{\text{imageWidth}}{2}\right)$
   - $\text{originY} = \left(\frac{\text{gridHeight}}{2} - \frac{\text{imageHeight}}{2}\right)$
2. **Nové souřadnice s posunem:**
   - $\text{newX} = \left(\frac{\text{gridWidth}}{2} - \frac{\text{imageWidth}}{2}\right) + \text{offsetX}$
   - $\text{newY} = \left(\frac{\text{gridHeight}}{2} - \frac{\text{imageHeight}}{2}\right)$

Pro konkrétní případ posunu o 256 px napravo:

- $\text{newX} = \left(\frac{\text{gridWidth}}{2} - \frac{\text{imageWidth}}{2}\right) + 256$
- $\text{newY} = \left(\frac{\text{gridHeight}}{2} - \frac{\text{imageHeight}}{2}\right)$

Dosazení hodnot:

- $\text{newX} = (128 - 73.1) + 256 = 54.9 + 256 = 310.9$
- $\text{newY} = 128 - 122.3 = 5.7$

Tímto způsobem budete mít obrázek přesně o 256 px napravo od původní pozice, stále ve středu na ose y.

Affinity software (affinity.html)

FAQ

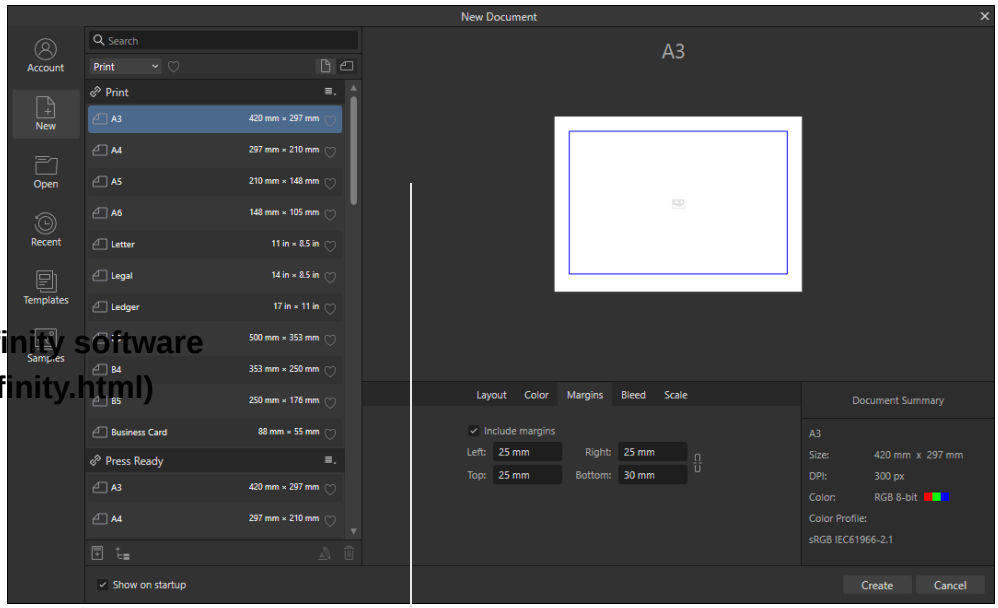
Modrý čtverec uvnitř stránky

Je způsoben odsazením. (Include margins)

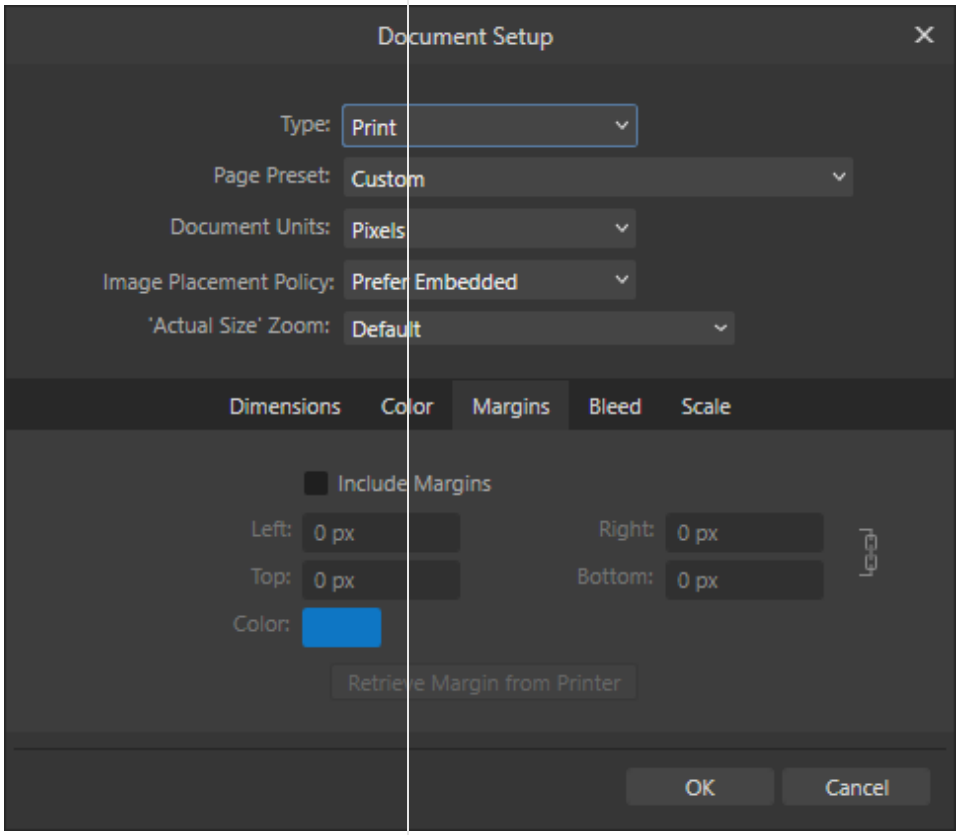
Řešení:

- Při vytváření nového dokumentu:

Affinity software (affinity.html)



- Pro otevřený dokument:



Dešifrování HTTPS (SSL/TLS)

1. Nastavit v proměnném prostředí SSLKEYLOGFILE:

   Nastavte systémovou proměnnou s názvem `SSLKEYLOGFILE` na cestu k souboru, kam bude prohlížeč ukládat klíče.

2. Konfigurace Wiresharku pro dešifrování SSL/TLS:

   `Edit > Preferences > Protocols > TLS`.

   Nastavte `Pre-Master-Secret log filename` na stejnou cestu, kterou jste použili pro `SSLKEYLOGFILE`.

3. Zachycení a analýza paketů:

   Spusťte zachycení paketů ve Wiresharku.

   Po ukončení zachycení paketů byste měli být schopni vidět dešifrovaný provoz.

Filtrovat by tag
WireShark (wireshark.html)

Zobrazení

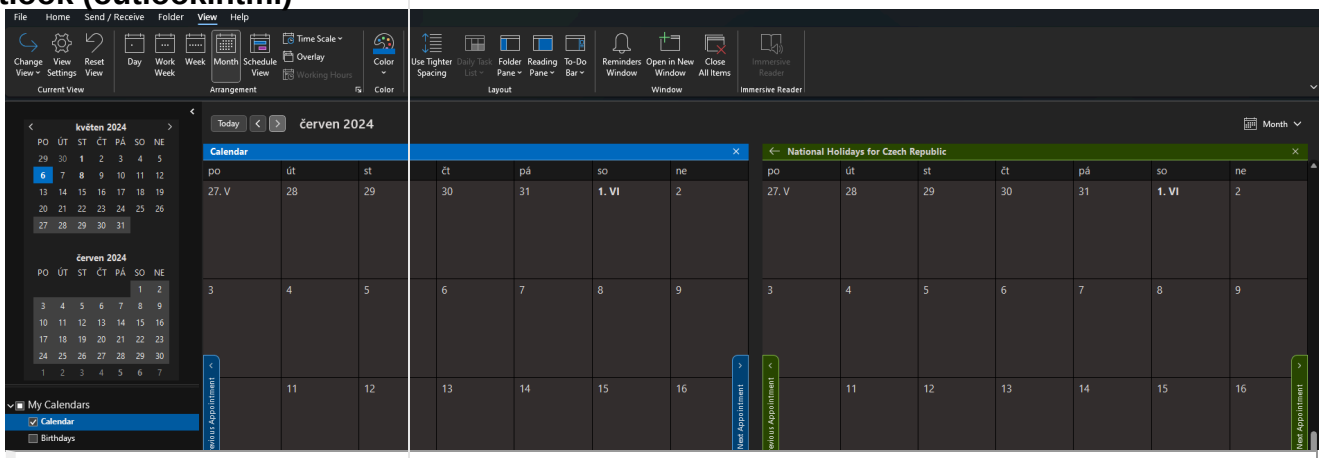
Sloučení/Oddělení panelů kalendáře

Filter by title

View ➡ Overlay

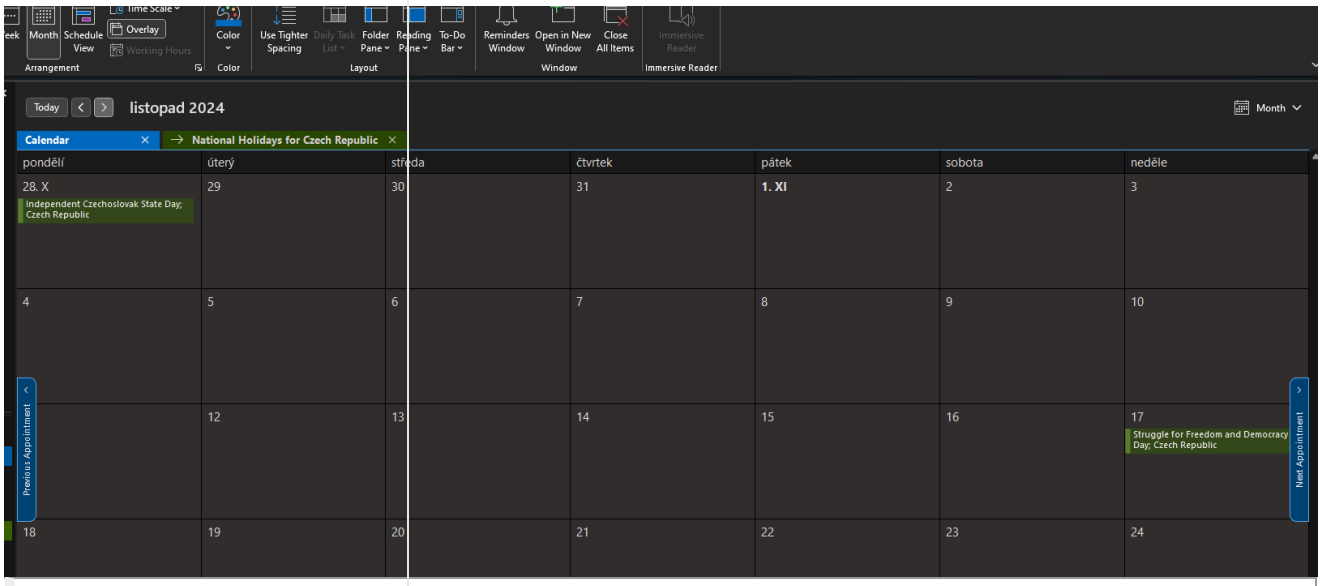
- Vypnuto

Outlook (outlook.html)



Každý kalendář se zobrazí v samostatném okně

- Zapnuto



Kalendáře se zobrazí v jednom okně

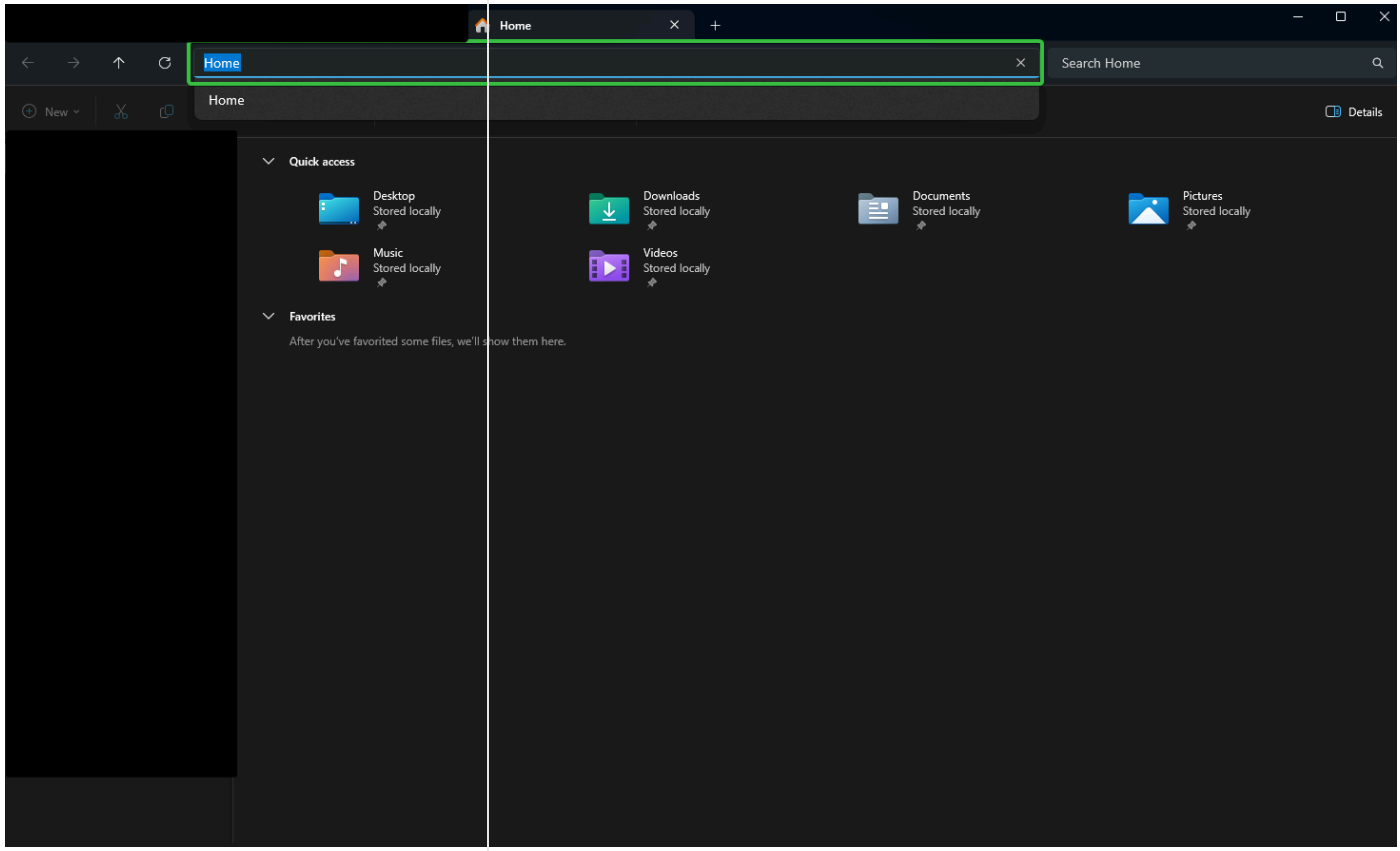
Klávesnicové zkratky

Minimalizace/Maximalizace všech oken

`win + D`

Skocení na adresní řádek

`Alt + D`



Chybějící klávesy na klávesnici

Kontextová klávesa

- **Windows (windows.html)**
- PowerShell (windows/powerShell.html)

Řešení:

```
Shift + F10
```